

forum.canton.network/t/9059

****[Feedback welcome]** OpenZeppelin's Daml development stack is public and open for review******

Archived 2026-09-13T23:49:00Z from ``https://forum.canton.network/t/9059.json``, the public Discourse endpoint. The untouched response is beside this file as ``forum-9059.json``; this is a rendering of it.

13 posts, 351 views.

1. Pepe_Blasco · 2026-08-12 14:48:59 UTC

Hi Canton community,

The first repos of OpenZeppelin's open-source Daml development stack for Canton are now public, and we're looking for your feedback. Test it, read the designs, and let us know how it works for what you're building, or where you would like to see improvements.

This release includes a Daml contracts library with an implementation of the CIP-112 token and settlement standard, and reference-implementation designs for the ecosystem's common financial use cases. It's built to the same standard as OpenZeppelin's libraries across Ethereum and other leading ecosystems, which have underpinned over \$35 trillion in onchain transfers.

Full details:

Token standard and settlement foundation

The specs repo consolidates a token template, a stablecoin reference, and the CIP-112 settlement primitive.

CIP-112 (Token Standard V2) is backward compatible with CIP-56 and adds settlement and further features.

The settlement primitive includes the atomic settlement logic the reference implementations build on, rather than each application reimplementing it. The interop exemplars for CIP-86, CIP-103, and CIP-104 can be validated against a LocalNet ledger over gRPC with `./scripts/localnet-cip-interop-validation.sh`, which demonstrates the implementations working together rather than in isolation.

Repos:

OpenZeppelin `canton-specs`
OpenZeppelin DAML template for CIP-056
OpenZeppelin DAML stablecoin

Reference-implementation designs

Architecture and design documents for the first four reference implementations, each a documented architecture for a specific financial application. These are design documents only; implementation code begins in the next milestone and is not present in these repos.

Privacy-preserving DEX: exchange design using atomic settlement as the core primitive, with a modular structure extending to AMM and other venue types.
Lending protocol: lending-market design built on the same settlement core.
Cross-chain stablecoin payments: stablecoin payment flow spanning Canton and external chains, with reserve and attestation controls.
Confidential auction launchpad: sealed-bid auction design using Canton's privacy

model.

Repos:

OpenZeppelin Canton Privacy-preserving DEX
OpenZeppelin Canton Institutional Lending protocol
OpenZeppelin Canton Crosschain stablecoin payments
OpenZeppelin Canton Confidential auction launchpad

Review and feedback are welcome

Clone the repos, read the design reports, and open an issue with how it works for what you're building, or where it should change.

Note that development is still in progress, and these components haven't been audited yet. All releases will undergo full security audit after collecting community feedback.

Pepe Blasco

Engineering Manager, OpenZeppelin

2. Erlan · 2026-08-13 16:00:40 UTC

The privacy-preserving DEX and cross-chain stablecoin payment designs immediately caught my attention. It's great to see reusable building blocks becoming available for the ecosystem. Thanks for sharing these resources with the community.

3. Mr_Tuddles · 2026-08-18 20:51:05 UTC

Our implementation of the new stack for our yielding and on-yielding stablecoins and agentic registry on Canton using this standard

GitLab

pearl-digital / p3-daml-contracts · GitLab

P3 Daml Contracts is the Canton Network implementation of the Pearl Path Protocol for Pearl Digital Treasury Company B.S.C.(c). It is the sibling of

4. zhe_bd · 2026-09-02 12:10:42 UTC

Awesome stuff! I noticed that all the DAML code are using AGPL license. Is that intentional?

5. woof-software · 2026-09-02 14:30:12 UTC

Thanks for putting these out for review. We spent most of our time in lending.md, and section 7 (Open Design Questions) is the part closest to our own work. On Compound we implemented the CAPO oracle for the correlated-collateral case, built the reserve-growth tracking, and we operate Compound Labs' Configurator pattern on the markets we deploy. Several of the section 7 questions are things we handle in production.

One observation from that side: oracle-induced risk and market risk fail differently and want separate controls. Your maxStaleness field already points that way. We wrote up the correlated-collateral case here: Correlated collateral and the parameter layer: notes for Canton vault builders

The question we'd like your read on is scope. Buffer sizing, debt ceilings, the change path for VaultParams, stress evidence for the insurance fund: is any of that planned for the reference implementations in M2 or M3, or does it stay a layer that implementers bring? We ask because that layer, parameter governance and calibration on Zenith, is what our open Dev Fund proposal covers (Proposal: Canton Risk Engine by Noosphere-314 · Pull Request #665 · canton-foundation/canton-dev-fund · GitHub). If you're already building it we'd rather know now. If you aren't, we'd rather compose with your stack than build something that sits awkwardly next to it.

Glad to go further on any of this, in whatever format is useful to you.

6. Pepe_Blasco · 2026-09-03 10:01:20 UTC

Thanks @woof-software, this is useful feedback.

For context, the reference implementations are not products we will deploy. They are complete, audited starting points that teams fork and operate. That draws the line like this:

We will be implementing the baseline parameter surface and its change path. VaultParams, maxStaleness, maxDeviation, the oracle interface requirements, role transfer through access control and ownable, and the SCU rules for adding fields such as a debt ceiling.

We do not own calibration. Buffer and grace-window sizing, ceiling values, insurance-fund stress evidence, and shock testing depend on the collateral, the market, and the operator. We document the questions in section 7 so that a fork knows what it must answer, but we do not plan to ship a risk engine or a simulator.

So your proposal does not overlap with ours. It fills the layer we leave to operators. What matters is that the two compose cleanly, and I see two seams:

Parameter governance. Your registry with propose-execute windows and execution-time validation is a natural author for VaultParams updates.

Oracle implementation. The RI specifies the PriceOracle interface and the requirements on the update mechanism, not the mechanism.

The current design is an initial architecture document. It will evolve as we get started the implementation, and that is the right point to align on the two seams above.

Thanks again for the interest!

7. Pepe_Blasco · 2026-09-03 10:04:27 UTC

Hey @zhe_bd, thanks for your interest!

The libraries (daml code) that we will be developing going forward are MIT license: GitHub - OpenZeppelin/canton-contracts: OpenZeppelin library for secure smart contract development on Canton · GitHub

The tooling systems will be AGPL license.

If you are interested in using the tools are have any conflict with the licensing, feel free to reach out and we can look at your usecase directly.

8. orumedominic · 2026-09-08 08:46:23 UTC

I took up the invitation to test these. I made each assertMsg/ensure condition vacuumous one at a time, rebuilt the DAR, and re-ran the suites: 52 of 69 fences can be removed with every test still passing - 3/7 in access-control-v1, 24/32 in simple-token, 25/30 in stablecoin.

Not vulnerabilities: every fence is present and correct. But you described these as audited starting points that teams fork and operate, and a fork inherits the suite - so this is what it will and won't catch when someone changes the code later.

Method, the controls, and the three defects in my own harness that produced false findings first: cantor8_kya_rails/docs/findings-openzeppelin.md at 2c4948ff3b25d2e24fda2f2ca7e8537d8ea57164 · dominicrume/cantor8_kya_rails · GitHub

Filed per repo: canton-contracts#43, canton-token-template#9, canton-stablecoin#9.

Happy to be told any of these are deliberately redundant or covered by a test my harness can't attribute.

9. orumedominic · 2026-09-10 21:32:07 UTC

You mentioned an agentic registry alongside the stablecoins.

Does anyone ever ask you to prove your agent didn't do something?

10. Mr_Tuddles · 2026-09-11 00:41:02 UTC

Yes, we support deterministic settlement by agents.

what do you have in mind?

11. orumedominic · 2026-09-11 03:30:52 UTC

I built a way to keep a record of what an agent tried to do, including the attempts that get blocked. Anyone can check it without trusting whoever wrote it.

What does deterministic settlement give you when an agent gets blocked?

12. Mr_Tuddles . 2026-09-11 09:54:47 UTC

Might be easier for you to just tell us what you are building.

We are a stablecoin issuer; our stables are deployed on etstnets with agentic payments.

pearldigital.com/agentic

We are investigating ways to port this to Canton natively, we have contracts deployed based on <https://gitlab.com/pearl-digital/p3-daml-contracts>

13. orumedominic . 2026-09-13 12:03:33 UTC

Fair. Plainly.

Your agentic registry says what an agent may do.

Deterministic settlement says what happens when it pays.

I do the third one. "I keep the record of what it tried and was refused."

You said you are porting to Canton natively. The gap gets wider there.

In Daml, a failed `assertMsg` aborts the transaction. So a blocked payment leaves nothing at all. Your settled payments sit on the ledger. Your blocked ones sit in your own logs, written by you, about you.

An auditor asks a stablecoin issuer for the second half.

So two changes. The refusal becomes a transaction that commits and moves no money. And refusals can be handed over without the payments.

Here is one:

https://dominicrume.github.io/cantor8_kya_rails/examples/settlement-refusals.json

Drop it on Know Your AgenticAI. The evidence layer under an agent platform.

It shows three refusals in full. Three payments stay shut. Delete a refusal and the file stops verifying. Nothing to install.

The figures are invented.

MIT, and about twenty lines if you would rather not take the dependency.
